

EXPERIMENT-32

Algorithm:

1. Start.

2. Display the menu:

Matrix addition — commutative property

Matrix multiplication — non-commutative property

Exit

3. Read the user's choice.

4. Use switch to perform the selected operation.

5. For matrix addition:

Input the dimensions.

Input matrices A and B.

Calculate $A + B$.

Calculate $B + A$.

Display both matrices.

Since they are equal, addition is commutative.

6. For matrix multiplication:

Input the order of two square matrices.

Input matrices A and B.

Calculate $A \times B$.

Calculate $B \times A$.

Compare the resulting matrices.

If they are different, it demonstrates that multiplication is not commutative.

7. Repeat the menu until Exit is selected.

8. Stop.

Example for Addition:

Suppose:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

$$B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

Then:

$$A + B = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}$$

$$B + A = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}$$

Therefore:

$$A + B = B + A$$

So, matrix addition is commutative.

Example for Multiplication:

For the same matrices:

$$A * B = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

$$B * A = \begin{pmatrix} 23 & 34 \\ 31 & 46 \end{pmatrix}$$

Therefore:

$$A \times B \neq B \times A$$

So, matrix multiplication is not commutative in general.

Program, Compilation and Execution:

```
GNU nano 9.0 32.c
#include <stdio.h>

// Function to input matrix elements
void inputMatrix(int r, int c, int a[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}

// Function to display a matrix
void displayMatrix(int r, int c, int a[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("%d\t", a[i][j]);
        }
        printf("\n");
    }
}

// Function to prove matrix addition is commutative
void matrixAddition()
{
    int r, c, i, j;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &r, &c);

    int A[r][c], B[r][c];
    int AB[r][c], BA[r][c];

    printf("Enter elements of matrix A:\n");
    inputMatrix(r, c, A);

    printf("Enter elements of matrix B:\n");
    inputMatrix(r, c, B);
}
```

```
GNU nano 9.0 32.c
// A + B
for (i = 0; i < r; i++)
{
    for (j = 0; j < c; j++)
    {
        AB[i][j] = A[i][j] + B[i][j];
    }
}

// B + A
for (i = 0; i < r; i++)
{
    for (j = 0; j < c; j++)
    {
        BA[i][j] = B[i][j] + A[i][j];
    }
}

printf("\nA + B:\n");
displayMatrix(r, c, AB);

printf("\nB + A:\n");
displayMatrix(r, c, BA);

printf("\nSince A + B = B + A,\n");
printf("Matrix addition is commutative.\n");
}

// Function to prove matrix multiplication is not commutative
void matrixMultiplication()
{
    int n, i, j, k;

    printf("Enter order of square matrices: ");
    scanf("%d", &n);

    int A[n][n], B[n][n];
    int AB[n][n], BA[n][n];

    printf("Enter elements of matrix A:\n");
    inputMatrix(n, n, A);

    printf("Enter elements of matrix B:\n");
    inputMatrix(n, n, B);
}
```

```
GNU nano 9.0 32.c
// Calculate A * B
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        AB[i][j] = 0;
        for (k = 0; k < n; k++)
        {
            AB[i][j] += A[i][k] * B[k][j];
        }
    }
}

// Calculate B * A
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        BA[i][j] = 0;
        for (k = 0; k < n; k++)
        {
            BA[i][j] += B[i][k] * A[k][j];
        }
    }
}

printf("\nA * B:\n");
displayMatrix(n, n, AB);

printf("\nB * A:\n");
displayMatrix(n, n, BA);

// Compare AB and BA
int same = 1;

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        if (AB[i][j] != BA[i][j])
        {
            same = 0;
        }
    }
}
}
```

```
GNU nano 9.0 32.c
{
    printf("\nA * B  $\neq$  B * A\n");
    printf("Hence, matrix multiplication is not commutative.\n");
}
else
{
    printf("\nFor these particular matrices, A * B = B * A.\n");
    printf("However, matrix multiplication is not commutative in general.\n");
}
}

int main()
{
    int choice;
    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Prove matrix addition is commutative\n");
        printf("2. Prove matrix multiplication is not commutative\n");
        printf("3. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                matrixAddition();
                break;

            case 2:
                matrixMultiplication();
                break;

            case 3:
                printf("Exiting program... \n");
                break;

            default:
                printf("Invalid choice! Enter 1, 2 or 3.\n");
        }
    } while (choice  $\neq$  3);

    return 0;
}
```

```
===== MENU =====
1. Prove matrix addition is commutative
2. Prove matrix multiplication is not commutative
3. Exit
Enter your choice: 1
Enter number of rows and columns: 2
2
Enter elements of matrix A:
1 2
3 4
Enter elements of matrix B:
5 6
7 8

A + B:
6      8
10     12

B + A:
6      8
10     12

Since  $A + B = B + A$ ,
Matrix addition is commutative.

===== MENU =====
1. Prove matrix addition is commutative
2. Prove matrix multiplication is not commutative
3. Exit
Enter your choice: 2
Enter order of square matrices: 2
Enter elements of matrix A:
2
1 2
3 4
Enter elements of matrix B:
5 6
7 8

A * B:
14      17
26      31

B * A:
18      19
26      27

 $A * B \neq B * A$ 
Hence, matrix multiplication is not commutative.
```

NAME: TAANISH AGARWAL
BATCH: B-15
SAP ID: 590034662